

How to use collections in Python

Summary

Python provides multiple collections. Each collection supports specific capabilities. Depending on the kind of information stored and retrieved, one collection is preferred over another. In this article, I share my knowledge on collections in Python. Programming with dictionaries is explained in depth.

Introduction

Python provides the below collections as part of the language:

1. List
2. Set
3. Tuple
4. Dictionary

Given a programming problem, choice of collection depends on the:

1. nature of the information stored.
2. order of storing and retrieval.
3. mutability.
4. computational complexity.

Collections

Below is a brief description of the four collections and when to use them.

List

A list

- stores a collection of similar items.

For example, to store a collection of names, use a list as shown below.

```
names = [ "bhaskar", "akshara", "laura", "bhaskar", "peter" ]
```

To store a collection of numbers, use a list as shown below:

```
numbers = [ 10, 20, 30, 40, 50 ]
```

- allows duplicate items to be stored; the “names” list above has two occurrences of “bhaskar.”

- stores elements in the order in which they are added.
- is mutable.

Set

A set

- stores a unique collection of items.
For example, to store a collection of unique colors, use a set as shown below.

```
colors = { "red", "blue", "green", "white", "black", "yellow" }
```

- does not allow duplicate items to be stored.
- does not store elements in the order in which they are added.
- is immutable.

Tuple

A tuple

- stores associated items together. For example, to store a pair of x and y coordinates of a point, use a tuple with two members as shown below:

```
point = (x_coordinate, y_coordinate)
```

- To store the dimensions of a box, use a tuple with three members representing the length, width, and height of the box as shown below:

```
dimensions = (20.5, 10.5, 10.5)
```

- allows duplicate items to be stored.
- stores elements in the order in which they are added.
- is immutable.
- is useful to return multiple values from a function.

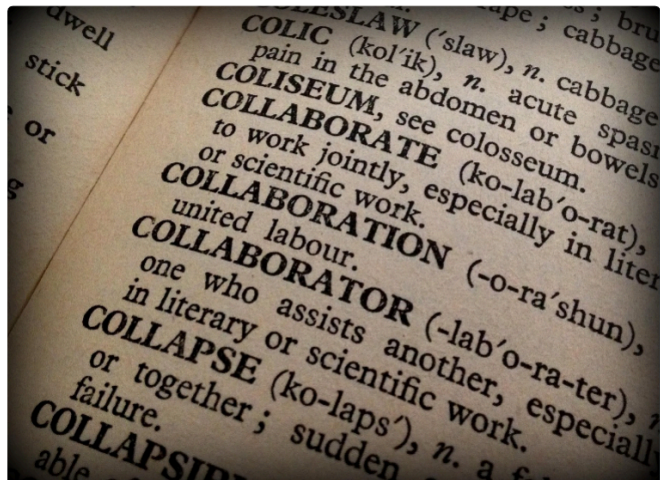
Dictionary

A dictionary

- stores key-value pairs.
A simple example could be a language dictionary storing words and their meanings. In the example below, the keys are words, and their meanings are values. An example dictionary is shown below:

```
word_dict = {
    "hazardous": "dangerous",
    "vociferous": "expressing your opinions or feelings in a loud and
confident way",
    "placebo": "a medicine or procedure prescribed for the psychological
benefit to the patient rather than for any physiological effect.",
    ...
}
```

- does not allow duplicate keys to be stored. However, duplicate values can be stored.
- stores elements in the order in which they are added.
- is mutable.



1. Key values stored in a dictionary are very similar to words and meanings storied in a language dictionary.
2. Retrieving keys from a dictionary is fast. This is similar to using the index pages in a dictionary.
3. All keys in a dictionary need to be of the same data type.

All keys stored in a dictionary must be of the same datatype. However, the values corresponding to each key can be of different datatypes, such as:

1. Strings
2. Integers
3. Floating point numbers
4. Booleans
5. Other collections: lists, tuples, sets, dictionaries

In the below dictionary, the values are of different data types:

```

bhaskar_dict = {
    "name": "bhaskar",
    "age": 54,
    "DOB": "23/06/1970",
    "weight": 67.2,
    "employed": False,
    "vehicles": [ "car", "car", "bike" ],
    "interests": { "maths", "biology", "computers", "infrastructure", "python"
},
    "vision": (-1.5, -1.25)
}

```

Comparison of collections

Below is a summary of the four collections in Python in tabular form.

	List	Set	Tuple	Dictionary
Elements	Mixed data types	Mixed data type	Mixed data types	Single data type for keys Mixed data types for values
Duplicates	Allowed	Not allowed	Allowed	Not allowed for keys Allowed for values
Ordering	Ordered	Unordered	Ordered	Ordered
Mutable	Mutable	Immutable	Immutable	Mutable
Worst-case search complexity	O(n)	O(n)	O(n)	O(1)

Dictionaries in detail

In this section, we look at dictionaries in detail. Below are code snippets to perform various operations on a dictionary.

1. To create a new dictionary:

```
students_branches = {  
    "akshara": "ise",  
    "aniket": "cs",  
    "aadya": "ise",  
    "riddhi": "ece",  
    "lakshmi": "ise"  
}
```

2. To insert a key-value pair into a dictionary:

```
students_branches["shraddha"] = "aiml"  
students_branches["srihari"] = "iem"
```

3. To retrieve a specific key-value pair from a dictionary:

```
print(students_branches["akshara"])
```

4. To update the value for a given key in a dictionary:

```
students_branches["akshara"] = "cs"
```

5. To remove a key-value pair from a dictionary:

```
del students_branches["aniket"]
```

6. To iterate over all elements in a dictionary:

```
for student in students_branches:  
    print(student)
```

7. Choice of storing information in a list or in a dictionary:

Dictionaries are an ideal choice when you want to group related information. In the below dictionary, all the key-value pairs are logically related to one person: "Bhaskar."

```
bhaskar_dict = {  
    "name": "bhaskar",  
    "age": 54,  
    "DOB": "23/06/1970",  
    "weight": 67.2,  
    "employed": False,  
    "vehicles": [ "car", "car", "bike" ],  
    "interests": { "maths", "biology", "computers", "infrastructure", "python"  
},  
    "vision": (-1.5, -1.25)  
}
```

If the same information above is stored using lists, it would look like the list below. In this case, "bhaskar_list" can store only the values, not the keys. The "keys" information is lost. As a result, while retrieving information from "bhaskar_list," the programmer needs to remember what each element in the list represents.

```
bhaskar_list = [ "bhaskar",  
    54,  
    "23/06/1970",  
    67.2,  
    False,  
    ["car", "car", "bike"],  
    { "maths", "biology", "computers", "infrastructure", "python" },  
    (-1.5, -1.25) ]
```

To workaround this limitation, a programmer can declare separate variables to store the individual items of the list and string them together to form the list, like shown below. However, the list still does not embed within it the "keys" and the mapping of the keys to their respective values.

```

bhaskar_dict = {
    name = "bhaskar"
    age = 54
    DOB = "23/06/1970"
    weight = 67.2
    employed = False
    vehicles = [ "car", "car", "bike" ]
    interests = { "maths", "biology", "computers", "infrastructure", "python" }
    vision = (-1.5, -1.25)
    bhaskar_list = [ name, age, DOB, weight, employed, vehicles, interests, vision ]
}

```

Programming problem

Consider the below programming problem:

You are provided student information for a class:

1. Name
2. Subjects
3. Tests for each subject
4. Marks scored in each test

The information is provided in a table like the below:

Marks	Mathematics			Chemistry		
Name	Test1	Test2	Test3	Test1	Test2	Test3
Akshara	80	60	50	90	60	70
Aadya	90	80	70	100	90	80
Aniket	90	80	70	90	80	70

You are asked to compute the average marks subject-wise for each student.

The above details can be stored in a list or a dictionary. However, a dictionary tends to be the better choice to store the information, as the “subject” keys will be lost if we store the information in a list.

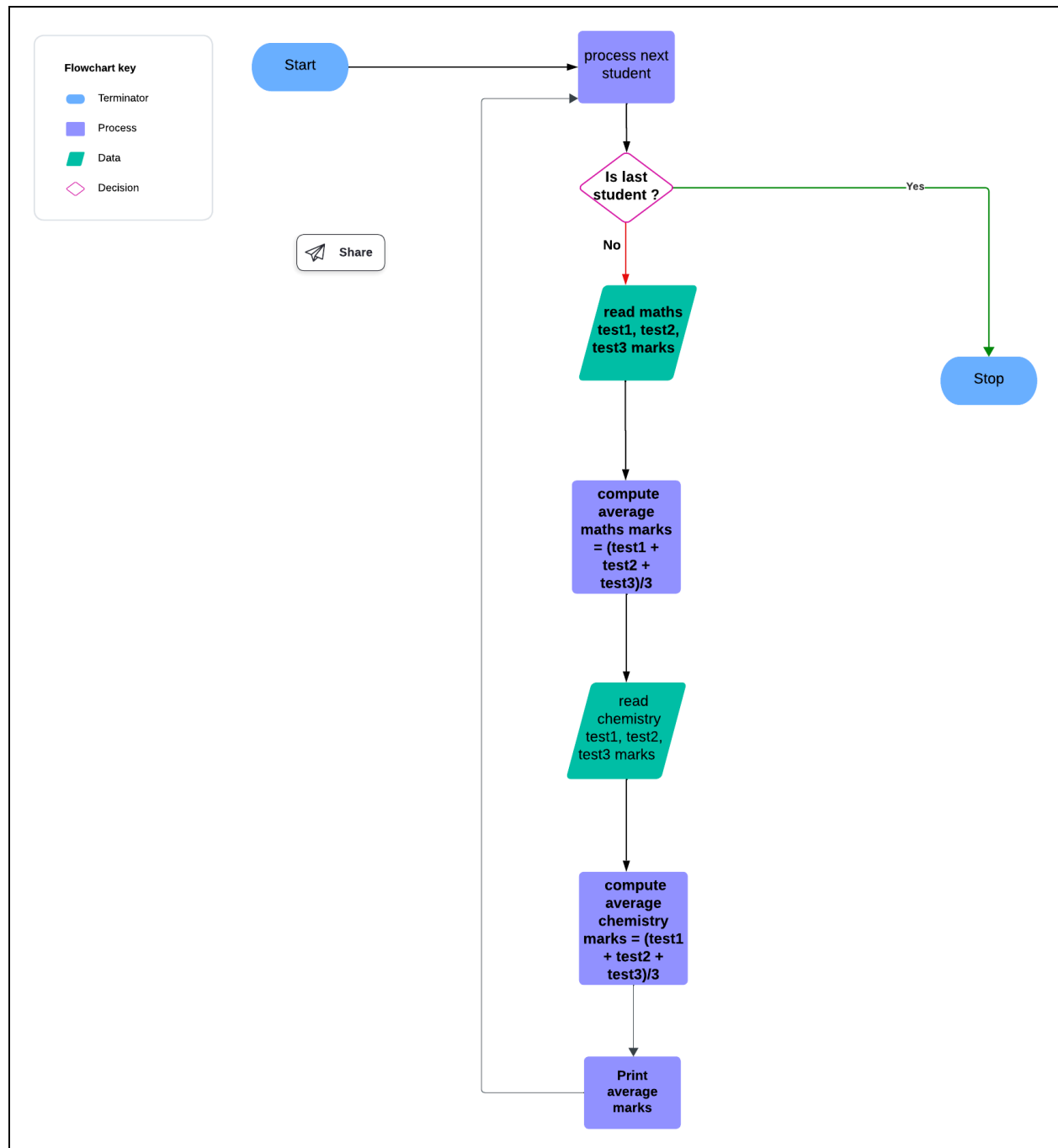
Below is the code snippet to store the information in a dictionary.

```

marks = {
  "akshara": {
    "maths": {
      "test1": 80,
      "test2": 60,
      "test3": 50
    },
    "chemistry": {
      "test1": 90,
      "test2": 60,
      "test3": 70
    }
  },
  "aadya": {
    "maths": {
      "test1": 90,
      "test2": 80,
      "test3": 70
    },
    "chemistry": {
      "test1": 100,
      "test2": 90,
      "test3": 80
    }
  },
  "aniket": {
    "maths": {
      "test1": 90,
      "test2": 80,
      "test3": 70
    },
    "chemistry": {
      "test1": 90,

```

The below flowchart shows how to read the marks subject-wise for each student and to compute the average marks.



Below is the code to compute the subject-wise average for each student. As you can see:

1. for each student;
2. for each subject;
3. marks are retrieved from the dictionary, and
4. average is computed and printed.

```
for student in marks:
    student_marks = marks[student]
    student_maths_marks = student_marks["maths"]
    student_chemistry_marks = student_marks["chemistry"]

    print(student)
    print("maths marks")
    print(student_maths_marks)
    mtest1 = student_maths_marks["test1"]
    mtest2 = student_maths_marks["test2"]
    mtest3 = student_maths_marks["test3"]
    avg_maths_marks = (mtest1 + mtest2 + mtest3) / 3
    print("avg_maths_marks = ",avg_maths_marks)
    print()
    print("chemistry marks")
    print(student_chemistry_marks)
    ctest1 = student_chemistry_marks["test1"]
    ctest2 = student_chemistry_marks["test2"]
    ctest3 = student_chemistry_marks["test3"]
    avg_chemistry_marks = (ctest1 + ctest2 + ctest3) / 3
    print("avg_chemistry_marks = ",avg_chemistry_marks)
    print()
```

Conclusion

Python provides four different collections:

1. List
2. Set
3. Tuple
4. Dictionary

Of these collections, lists and dictionaries are widely used to solve programming problems. Depending on the problem and the nature of the information involved, choose the right collection.